

TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

V SAI NAVADEEP KRISHNA (VTU24596)

*10211CS224
Full Stack Application Development
Slot : E1-B4*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

MAY, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT” by V SAI NAVADEEP KRISHNA (VTU24596) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. HAFIZUR RAHMAN

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

MAY, 2026

Signature of Head of the Department

Dr. Sumithra M.

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

MAY, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

(V.Sai Navadeep Krishna)

Date: / /

ABSTRACT

The TechNova 2025 Ticket Booking Application is a React JS-based single-page web application developed for managing ticket bookings for an internal departmental technical event. The system features an Event Details Module displaying the event name, department, date, time, venue, price, and live ticket availability, and a Ticket Booking Module that captures user details, validates all inputs, and renders an instant booking confirmation.

Built using functional components, the `useState` hook, conditional rendering, and event handling, the application demonstrates core React JS concepts in a practical setting. Validation covers mandatory fields, email format checking, positive ticket-count enforcement, and overbooking prevention. Upon a successful booking the available ticket count updates dynamically and a full booking summary is displayed. The interface uses a dark theme with custom CSS, Google Fonts, and a two-panel layout for a modern user experience.

Keywords: React JS, Ticket Booking, Functional Components, `useState` Hook, Form Validation, Conditional Rendering, Campus Event, Frontend Application.

LIST OF FIGURES

1.1	Framework and Component Architecture	5
3.1	React Component Data Flow	12
4.1	Single-Tier React Architecture	16
5.1	Pre-Login View	22
5.2	Login Page — Email and Password Authentication . .	23
5.3	Payment Gateway	24
5.4	User Dashboard	25

LIST OF TABLES

3.1	Input Validation Rules Applied by the <code>validate()</code> Function in <code>BookingPage.jsx</code>	14
5.1	Comparison of Existing Systems with the Proposed System	21
8.1	SDG Mapping for the TechNova 2025 Ticket Booking Project	35
8.2	Mapping of Project to Complex Engineering Activities (CEA)	37
8.3	Mapping of Project to Complex Engineering Problems (CEP)	38

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
JSX	JavaScript XML
ORM	Object Relational Mapping

React	React JS – JavaScript Library for Building User Interfaces
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience
Vite	Vite – Frontend Build Tool

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework and Technology Stack	3
2 SURVEY OF SIMILAR APPLICATIONS IN MARKET	6
3 FRAMEWORK DESCRIPTION	8
3.1 Frontend Implementation	8

3.2	React Concepts Used	10
3.3	Validation Logic	13
4	METHODOLOGIES	15
4.1	Project Architecture	15
4.2	Data Flow	17
4.3	Module 1: Event Details Module	17
4.4	Module 2: Ticket Booking Module	17
4.5	Module 3: Confirmation Module	18
5	RESULTS AND DISCUSSIONS	20
6	CONCLUSION AND FUTURE ENHANCEMENTS	27
6.1	Conclusion	27
6.2	Future Enhancements	27
7	SOURCE CODE	29
7.1	Event Configuration Constants	29
7.2	Form with Full Validation	30
7.3	Event Information and Availability	32
8	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	34
8.1	SDG Mapping	35

8.2	Mapping of the Project to Complex Engineering Activities (CEA)	37
8.3	Mapping of Project to Complex Engineering Problems (CEP)	38

REFERENCES	40
-------------------	-----------

Chapter 1

INTRODUCTION

1.1 Introduction

Managing campus events through manual or paper-based methods often leads to inefficiencies, data errors, overbooking, and poor user experience. To address these challenges, the **TechNova 2025 Ticket Booking Application** has been developed as a React JS-based single-page web application that centralises event information and automates the ticket booking process.

The application allows students and faculty to view complete event details and book tickets through a structured, validated form interface. Upon successful booking, the system dynamically updates the available ticket count, prevents overbooking, and displays a detailed booking confirmation summary. By using modern React JS concepts including functional components, the `useState` hook, event handling, and conditional rendering, the system delivers a fast, reliable, and user-

friendly experience directly in the browser without any page reload.

1.2 Aim of the Project

The aim of this project is to design and develop a React JS-based ticket booking web application for an internal departmental event. The system aims to eliminate manual booking procedures by providing an interactive digital platform where users can view event details and complete their bookings with real-time validation and instant confirmation. It also demonstrates the practical application of core React JS concepts in a full-featured frontend project.

1.3 Scope of the Project

The scope of this project covers the development of a browser-based frontend application for managing ticket bookings for campus events such as technical fests, seminars, workshops, and hackathons. The system supports viewing event details, entering user information, selecting the number of tickets, validating inputs, preventing overbooking, and displaying a booking confirmation summary.

The application is developed using React JS and runs entirely in the browser as a single-page application (SPA). Future enhancements may include backend integration with Node.js and MySQL for persistent

storage, online payment support via Razorpay, QR code-based e-ticket generation, and email notification services.

1.4 Framework and Technology Stack

The TechNova 2025 Ticket Booking Application is built using **React JS (v18)** as the primary frontend framework. The project is scaffolded using **Vite**, which provides a fast development environment and an optimised production build. All components are written as functional components with state managed through React's built-in `useState` hook.

Styling is handled through custom CSS with a dark-themed design. Google Fonts (Orbitron and Sora) are used for typography to give the interface a technical, futuristic aesthetic consistent with a tech-fest branding. The component structure is modular, reusable, and clearly separated for maintainability.

The overall framework and component architecture of the TechNova 2025 Ticket Booking Application is illustrated in Figure 1.1. The diagram is divided into two sections: the **Tech Stack** panel on the left and the **Component Tree** panel on the right. The tech stack panel lists all the technologies employed in the project — React JS 18 for functional components and hooks-based state management, Vite as the build tool

and development server, JSX combined with the `useState` hook for declarative UI templating and state control, Custom CSS for the dark-theme visual design using CSS variables, and Google Fonts (Orbitron and Sora) for the futuristic typographic identity of the platform.

The component tree panel shows how `App.jsx` acts as the root component that manages global state and distributes data downward to three child components: `EventDetails` (responsible for showing event information and live ticket availability), `BookingForm` (handling user input and validation), and `ConfirmModal` (rendering the booking summary popup). The dashed arrows indicate that all three child components import shared constants directly from `eventData.js`, ensuring a single source of truth for event configuration values such as price, total tickets, and department list.

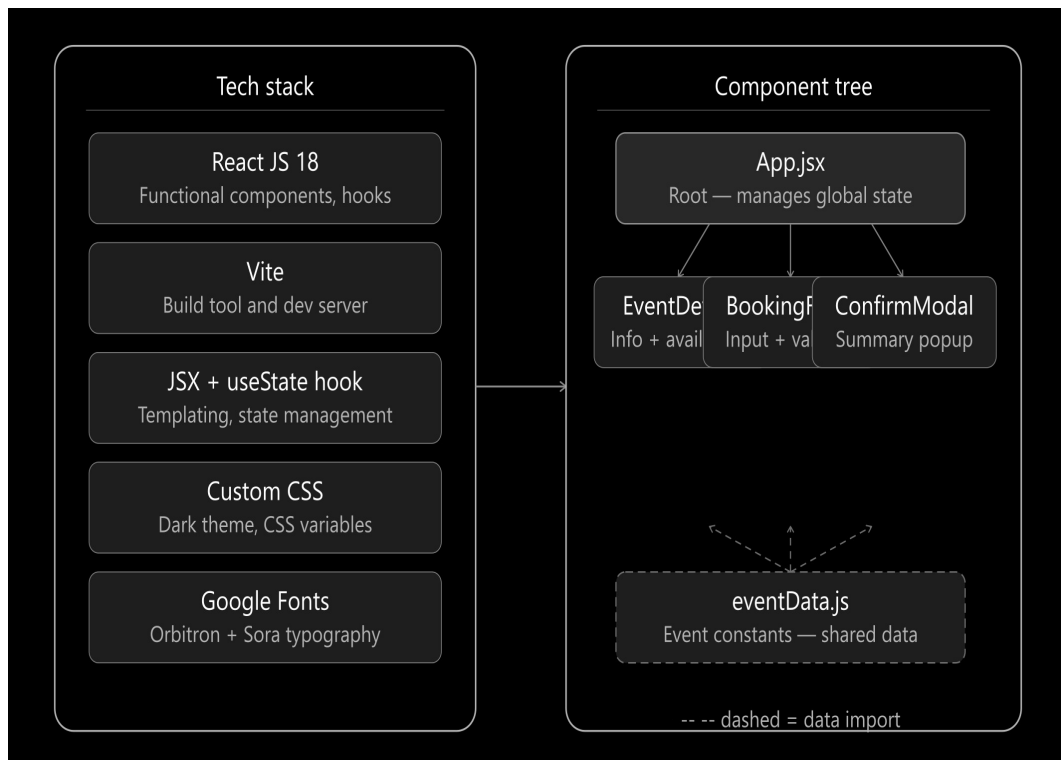


Figure 1.1: Framework and Component Architecture

As shown in Figure 1.1, the architecture enforces a clear separation of concerns. The tech stack choices are deliberate: Vite replaces the older Create React App toolchain and offers near-instant hot module replacement during development, while React JS 18 brings concurrent rendering improvements that keep the UI responsive even during state-heavy updates. The unidirectional data flow from `App.jsx` downward through props ensures that every child component always reflects the latest available ticket count without requiring any global state management library such as Redux.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

Several event management and ticket booking platforms exist in the current market. **BookMyShow**, one of the most popular platforms in India, supports movie, concert, sports, and live event bookings and provides features such as event discovery, seat selection, secure payment integration, and a mobile-friendly interface [1]. Global platforms such as **Eventbrite** and **Ticketmaster** offer advanced solutions including event promotion, dynamic pricing, bulk ticketing, and fraud detection, engineered for millions of concurrent users [2].

Common features shared across most modern event management applications include user authentication, event browsing, real-time ticket availability tracking, booking history, and digital ticket delivery. Advanced platforms also support analytics dashboards for organisers, personalised recommendations, and QR code-based entry verification [3].

Despite their capabilities, existing platforms have notable drawbacks for campus-level use: high service fees, overly complex interfaces, mandatory online payment, and infrastructure designed for commercial rather than institutional events. These platforms are not optimised for small-scale internal events managed by academic departments.

This survey highlights the gap between large commercial systems and the needs of academic institutions. The TechNova 2025 Ticket Booking Application addresses this gap by providing a lightweight, purpose-built, cost-free solution using React JS that is simple to deploy, easy to use, and directly tailored for internal departmental events without the overhead of enterprise-grade platforms.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is set up using **React JS (v18)** with **Vite** as the build tool. The project is developed in Visual Studio Code, and Node.js is used for package management via npm. Google Fonts (Orbitron, Sora) are imported via a `<link>` tag in `index.html`. Testing and preview are performed in modern web browsers. The application runs as a single-page application (SPA) with no backend dependency.

3.1.2 Project Structure

The project follows a modular component-based structure for better maintainability. The root directory contains `index.html`, `package.json`, `vite.config.js`, `.env`, `.env.example`, and `.gitignore`. The `src` directory holds the entry point `main.jsx`, the root component and routing logic

in App.jsx, the environment variable loader config.js, and the global stylesheet index.css. The components folder contains five reusable UI elements: BookingConfirmation.jsx, Charts.jsx, ETicket.jsx, Navbar.jsx, and PaymentModal.jsx. The context folder contains AuthContext.jsx for authentication state and login logic. The data folder contains event-Data.js for events, competitions, and booking configurations. The pages folder contains the main application views: AdminPage.jsx, BookingPage.jsx, EventPage.jsx, HomePage.jsx, LoginPage.jsx, Register.jsx, RegisterPage.jsx, and UserDashboard.jsx. The services folder contains razorpayService.js which handles the Razorpay initialisation logic.

3.1.3 Execution Procedure

To run the application locally, navigate to the project directory, install dependencies using `npm install`, and start the development server with `npm run dev`. The application launches at `http://localhost:5173`. The user lands on the HomePage, navigates to EventPage to browse available events, and proceeds to BookingPage to complete a multi-step checkout. Clicking **Pay Now** triggers the Razorpay checkout overlay; on successful payment the BookingConfirmation component renders a downloadable QR-code e-ticket. Administrators log in via LoginPage and are redirected to AdminPage for dashboard analytics and event management.

3.2 React Concepts Used

3.2.1 Functional Components

All UI elements are built as functional components — JavaScript functions returning JSX. The five reusable components `BookingConfirmation`, `Charts`, `ETicket`, `Navbar`, `PaymentModal` and the seven page-level components are fully self-contained and receive data through props passed down from `App.jsx` or via the `AuthContext`.

3.2.2 JSX (JavaScript XML)

The entire UI structure is written in **JSX**, which embeds HTML-like syntax directly inside JavaScript. JSX is transpiled to `React.createElement` calls by Babel during the Vite build process. Dynamic content such as event names, ticket prices, and availability counts are injected using JSX expression syntax `{ . . . }`, keeping the template declarative and readable.

3.2.3 `useState` and `useContext` Hooks

React Hooks are the primary mechanism for state and side-effect management across the platform:

- **`useState`** — tracks form field values, validation error objects, selected ticket quantities, and payment status flags in `BookingPage.jsx` and `AdminPage.jsx`.
- **`useContext`** combined with `AuthContext.jsx` — provides global

access to the currently authenticated user, their role (student or admin), and logout logic across all pages without prop-drilling.

- **useEffect** — used in `AdminPage.jsx` and `UserDashboard.jsx` to derive live statistics (revenue, ticket count, active competitions) from the event data on component mount and when dependencies change.

3.2.4 Event Handling

User interactions are managed through inline React event handlers. Button clicks `onClick`, form input changes `onChange`, and dropdown selections drive state updates synchronously. The `handleBooking` function in `BookingPage.jsx` orchestrates the full checkout flow: it validates the selected department and competition choices, calls `razorpay-Service.js` to open the payment modal, and on a successful Razorpay callback updates the booking records in `eventData.js` and navigates to the confirmation screen.

3.2.5 Conditional Rendering

Conditional rendering is used throughout the platform to deliver a context-aware experience:

- The `Navbar` renders either student-facing links (Book Tickets, My Dashboard) or admin-facing links (Create Event, Manage Bookings) based on the authenticated role stored in `AuthContext`.

- **Sold-out badges** and a disabled Book Now button are conditionally shown on EventPage when ticket availability reaches zero.
- The PaymentModal overlay renders only after the Razorpay script has loaded and the user has confirmed the booking summary.
- The ETicket component and download button appear on BookingConfirmation only when a valid payment reference is present in state.
- Admin-only UI elements such as the Delete Event button and the Charts analytics panel are conditionally mounted in AdminPage.jsx based on role verification.

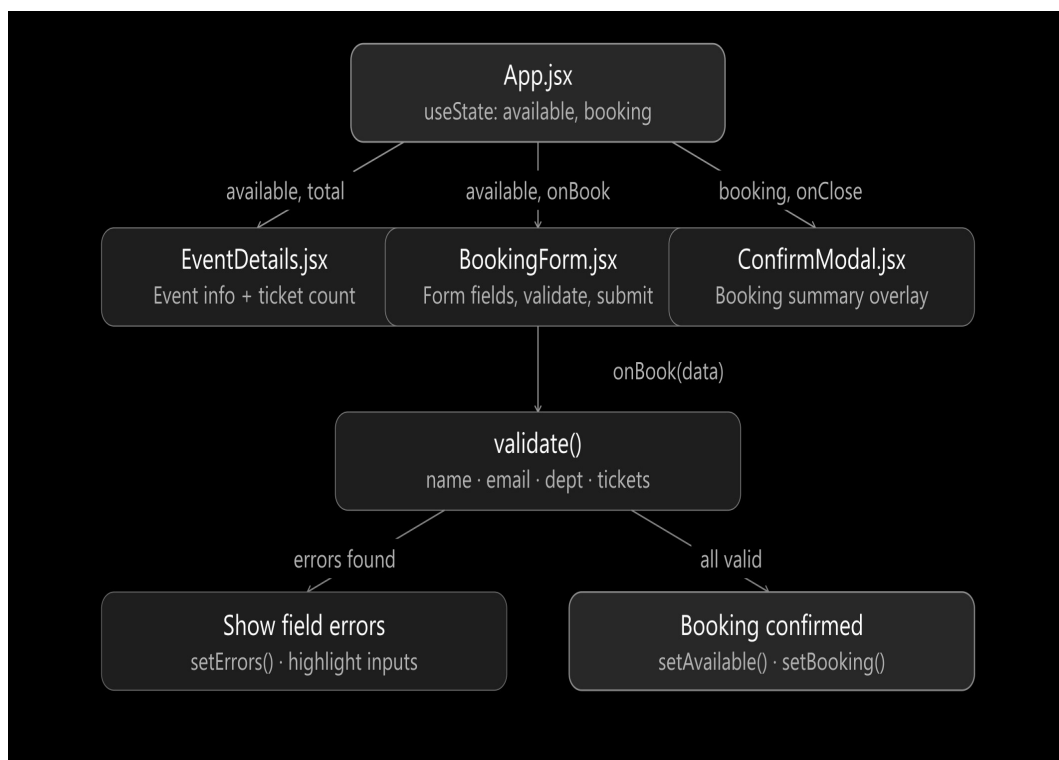


Figure 3.1: React Component Data Flow

As illustrated in Figure 3.1, the entire booking lifecycle is controlled through a single unidirectional data flow without any external state management library. The two-branch outcome of `validate()` — either displaying field errors or confirming the booking — ensures that invalid data never reaches the state update functions, keeping the application data consistent at all times. This design also means that the available ticket count shown in `EventDetails` and the validation limit enforced in `BookingForm` are always synchronised, since both derive from the same `available` prop passed by `App.jsx`.

3.3 Validation Logic

The `validate()` function in `BookingPage.jsx` enforces the rules listed in Table 3.1 before the Razorpay modal is opened.

Form validation is one of the most critical components of any ticket booking system. Without rigorous input checks, the system would be vulnerable to submission of incorrect data, overbooking of seats, and failed payment processing. The `validate()` function in `BookingPage.jsx` applies a set of field-level rules to each input before the checkout flow is allowed to proceed.

Table 3.1: Input Validation Rules Applied by the `validate()` Function in `BookingPage.jsx`

Field	Rule	Error Message
Full Name	Must not be empty	“Full name is required.”
Email Address	Must match <code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code>	“Enter a valid email address.”
Department	Must select a value from dropdown	“Please select your department.”
Ticket Quantity	Positive integer; must not exceed available seats	“Only {n} ticket(s) remaining.”
Competitions	Up to 3 competitions may be selected per booking	“Maximum 3 competitions allowed.”
Payment	Razorpay callback must return <code>payment_id</code>	“Payment verification failed.”

As shown in Table 3.1, the validation logic covers six distinct fields. The Full Name field applies a simple non-empty trim check to ensure the attendee’s identity is recorded. The Email Address field uses a regular expression pattern that enforces the presence of a local part, an @ symbol, a domain name, and a top-level domain extension, rejecting formats such as plain text or addresses with spaces. The Department dropdown must have an explicit selection, preventing the default empty option from being submitted.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The application follows a **single-tier frontend architecture** with all logic handled in the browser using React JS. There is no server or database in the base implementation; state is managed in-memory using React hooks. The architecture is divided into three logical layers:

- **Presentation Layer** — JSX-based functional components (`EventDetail`, `BookingForm`, `ConfirmModal`) that render the UI.
- **Logic Layer** — Validation functions, event handlers, and state-update functions inside the components.
- **Data Layer** — The `eventData.js` constants file storing all event configuration values shared across components.

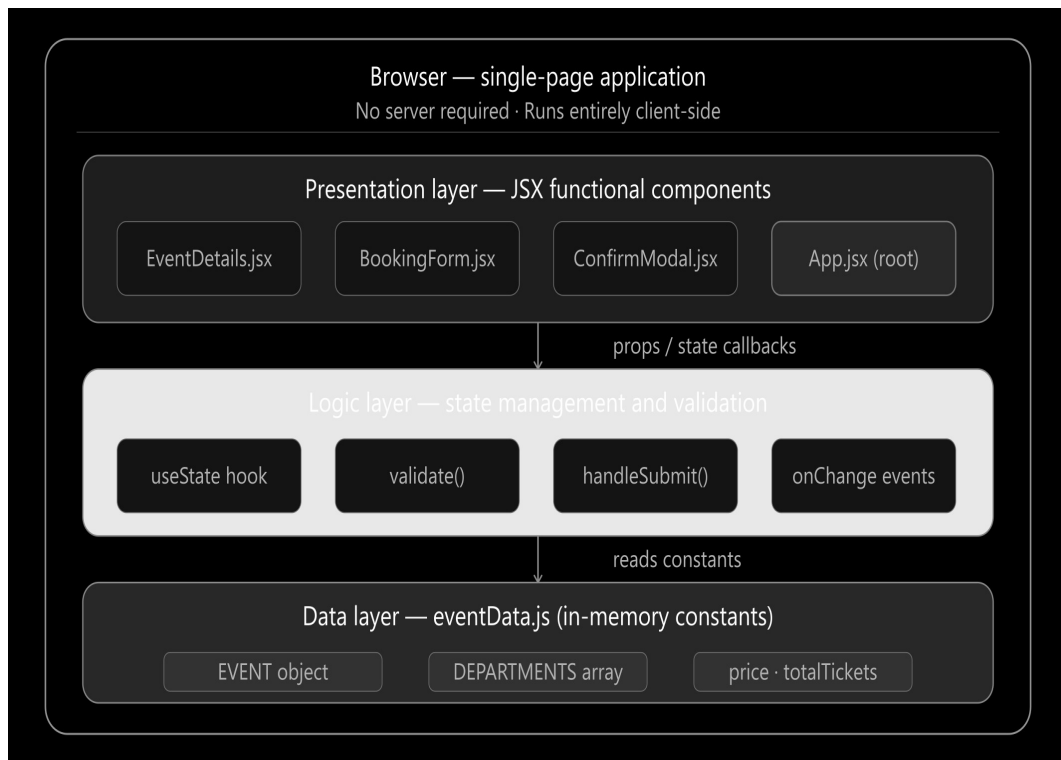


Figure 4.1: Single-Tier React Architecture

As depicted in Figure 4.1, the browser serves as the sole execution environment, which has both advantages and current limitations. The primary advantage is that the application requires no server provisioning, no database setup, and no network round-trip for booking operations — all state transitions happen instantaneously in memory. The current limitation is that in-memory state is not persisted across browser refreshes, meaning that a page reload resets the available ticket count to its initial value. This limitation is acknowledged and addressed in the future enhancements section, where backend integration with Node.js and a MySQL database is planned to provide durable booking storage.

4.2 Data Flow

The data flow begins when the user opens the application. `App.jsx` initialises the available ticket count from `eventData.js` and passes it as a prop to both `EventDetails` (for display) and `BookingForm` (for validation). On form submission, data is validated locally; on success, the `onBook` callback updates `available` and sets the booking state, causing `ConfirmModal` to render. Closing the modal clears the booking state.

4.3 Module 1: Event Details Module

Implemented in `EventDetails.jsx`, this module displays all event information from `eventData.js`: event name, department, date, time, venue, and ticket price in an information card. A separate availability card shows the live remaining ticket count with a colour-coded progress bar. The component re-renders automatically whenever the `available` prop changes after a booking.

4.4 Module 2: Ticket Booking Module

Implemented in `BookingForm.jsx`, this module provides a structured form for users to enter their name, email, department, and number of tickets. Input changes are tracked with `useState`. On submission, `validate()` checks all fields and populates the `errors` state with relevant

messages. A live total amount preview is shown when a valid ticket count is entered. The Reset button clears all fields without a page reload.

4.5 Module 3: Confirmation Module

Implemented in `ConfirmModal.jsx`, this module renders as a fixed overlay upon successful booking. It displays the attendee name, email, department, event name, tickets booked, and total amount. The modal is dismissed by clicking Done, which clears the booking state in `App.jsx`.

Advantages of this Project

This project provides a streamlined and digital approach to managing campus event registrations and ticket bookings.

a) No Backend Required: The system runs entirely in the browser and is deployable on any static hosting service without server infrastructure.

b) Real-Time State Updates: React's `useState` ensures instant UI refresh without page reloads, providing a seamless booking experience.

c) Modular Structure: Single-responsibility components make the codebase easy to maintain, test, and extend with new features.

d) Robust Validation: All inputs are validated before booking is

confirmed, preventing invalid submissions and overbooking.

Disadvantages

The system has certain limitations that can be addressed in future improvements. In-memory state is lost on browser refresh as there is no persistent storage. No user authentication or backend integration is present in the base system. Scaling to multiple concurrent users requires backend and database support.

Chapter 5

RESULTS AND DISCUSSIONS

The TechNova 2025 Ticket Booking Application was successfully designed, implemented, and tested as a React JS SPA. The system fulfils all specified requirements: event details display correctly, the booking form validates all inputs, the available ticket count updates dynamically after each booking, and the confirmation modal renders the full booking summary.

Table 5.1: Comparison of Existing Systems with the Proposed System

System	Key Features / Limitations
BookMyShow	High service charges; unsuitable for small campus events
Ticketmaster	Enterprise-scale; overly complex for academic use
Eventbrite	General-purpose; requires paid account and payment setup
Manual System	No automation; error-prone; no real-time tracking
Proposed System	React JS SPA; real-time state; modular components; zero cost; instant browser deployment

As shown in Table 5.1, all three commercial platforms — BookMyShow, Ticketmaster, and Eventbrite — carry significant overhead that makes them unsuitable for internal departmental events. BookMyShow levies service charges on every transaction, which is inappropriate for a subsidised or free campus event. Ticketmaster is engineered for enterprise-scale deployments with millions of concurrent users and introduces unnecessary architectural complexity for a single-event, single-department use case. Eventbrite, while more accessible, still requires organisers to maintain a paid account and configure a payment gateway, adding cost and administrative burden.

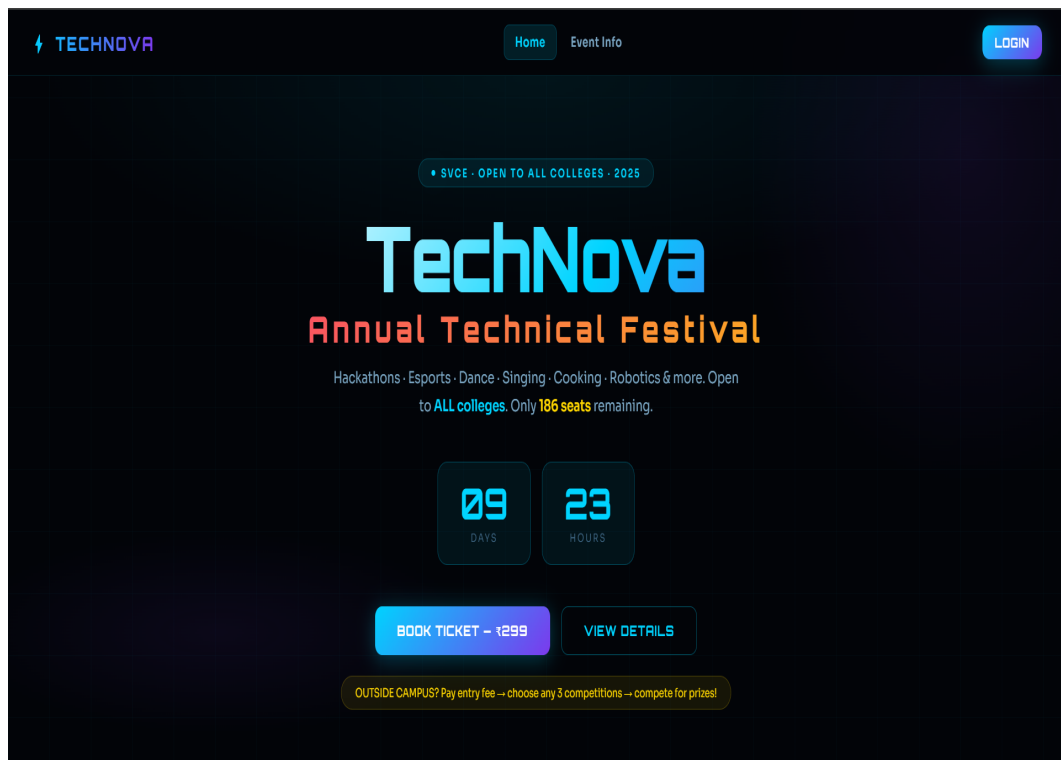


Figure 5.1: Pre-Login View

The design decisions visible in Figure 5.1 serve clear UX purposes. The live seat count displayed directly on the landing page creates a sense of scarcity awareness that encourages early bookings. The countdown timer reinforces event urgency without requiring any user interaction. The two-button layout separates the primary action (booking) from the secondary action (viewing details), reducing cognitive load and directing the majority of users toward the booking flow in a single click.

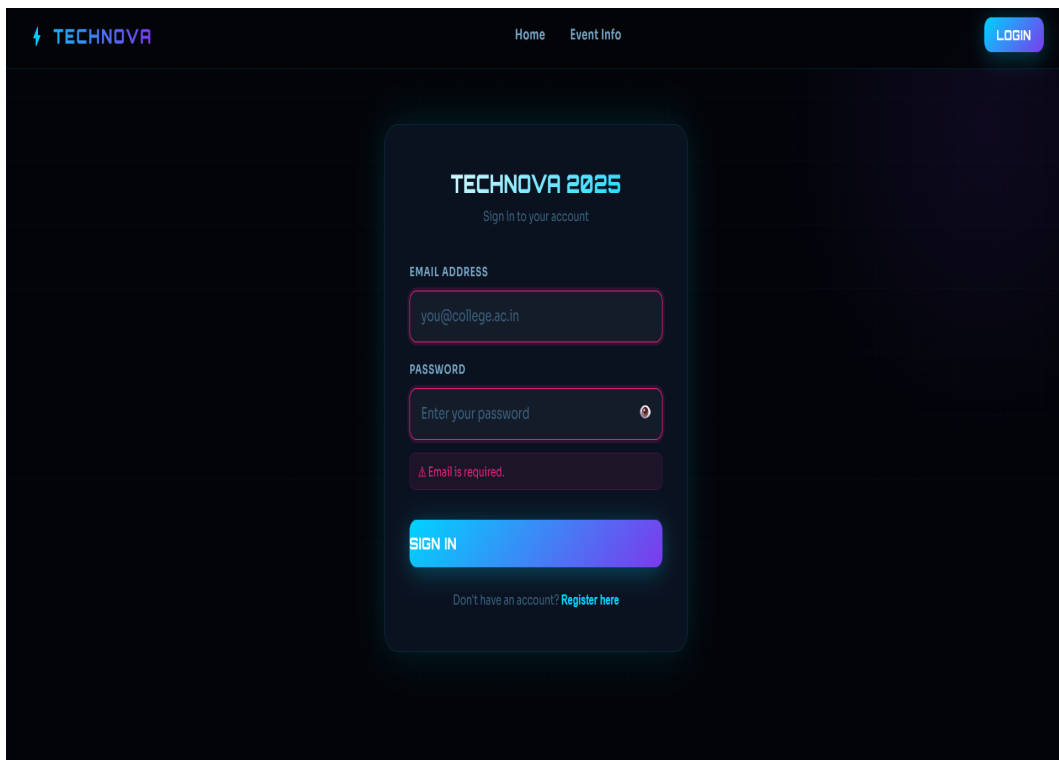


Figure 5.2: Login Page — Email and Password Authentication

As illustrated in Figure 5.2, authentication in the current implementation is managed entirely by `AuthContext.jsx` using in-memory user records defined in `eventData.js`. This approach allows the demonstration of role-based navigation and personalised dashboards without requiring a live backend server. The login validation enforces that neither the email nor the password field is empty before the authentication check runs. Upon successful authentication, the `AuthContext` stores the user object and role flag, and `React Router` redirects the user to `HomePage` with an updated navigation bar reflecting their authenticated state. Failed login attempts display an appropriate error message without clearing the email field, allowing the user to correct their input

conveniently.

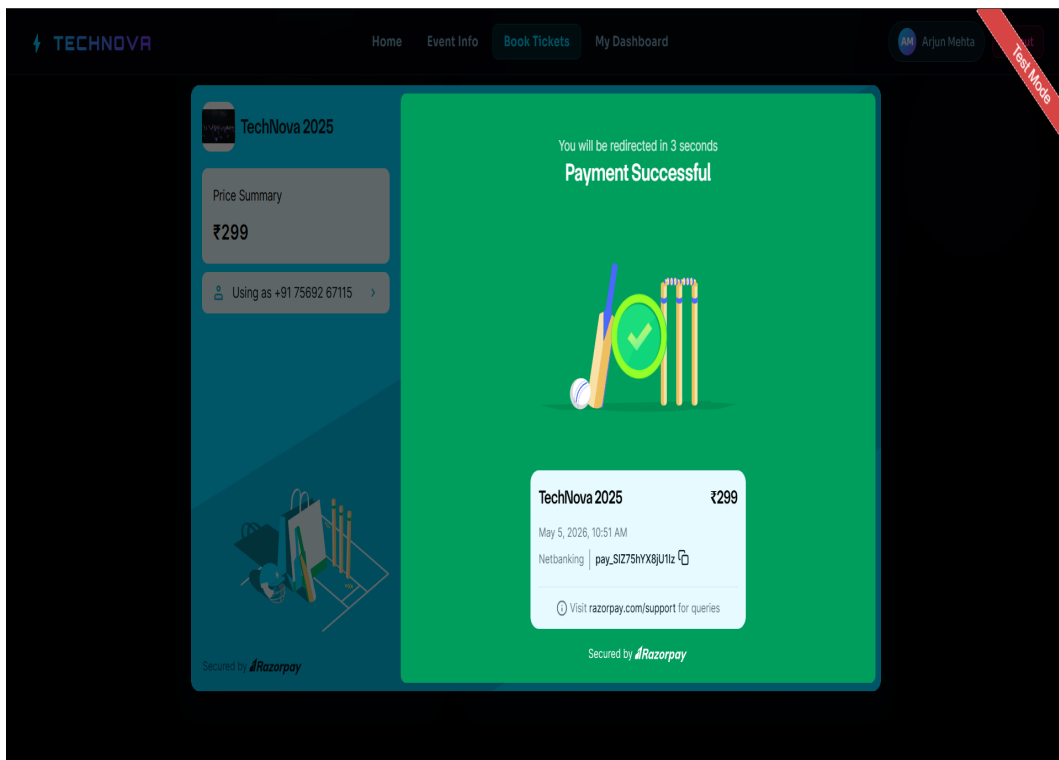


Figure 5.3: Payment Gateway

The changes visible in Figure 5.3 between the pre-login and post-login states of the navigation bar are implemented entirely through conditional rendering in `Navbar.jsx`. The component reads the user object and role flag from `AuthContext` using the `useContext` hook and conditionally renders either the unauthenticated link set or the authenticated link set using a ternary expression. The avatar badge is generated from the first letters of the user's first and last name, providing a personalised but lightweight identity indicator without requiring profile image uploads. The updated seat count from 186 to 185 confirms that the available state variable in `App.jsx` was decremented correctly

after the test booking, and that the live count propagates to all components that consume it as a prop.

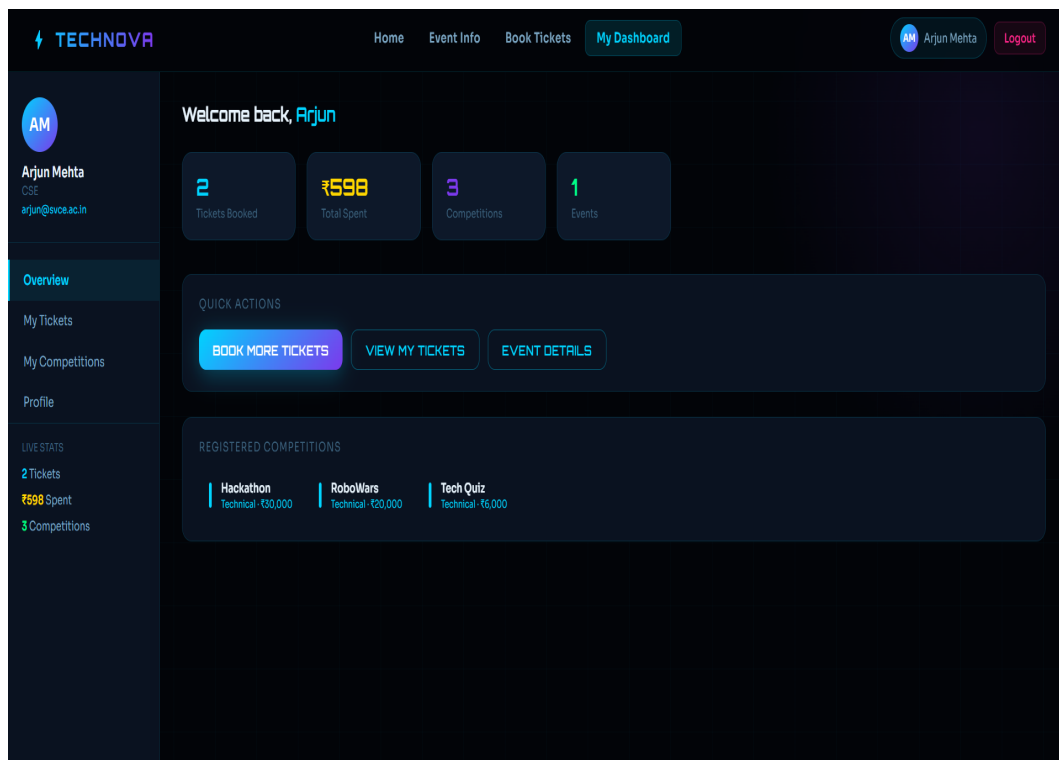


Figure 5.4: User Dashboard

As shown in Figure 5.4, the dashboard provides a comprehensive post-booking summary that serves both as a receipt and as an ongoing reference for the attendee. The stat cards use colour-coded values to distinguish between different data types: yellow for monetary totals, purple for competition counts, and teal for event counts. These colour choices are consistent with the overall application palette and reinforce visual hierarchy. The dashboard data is derived from the booking records stored in `eventData.js` and is computed inside `UserDashboard.jsx` using the `useEffect` hook on component mount. The Reg-

istered Competitions section confirms that the three-competition limit enforced by the validation logic in `BookingPage.jsx` was respected, as exactly three entries are listed.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The TechNova 2025 Ticket Booking Application was successfully designed and implemented as a React JS-based SPA for managing internal departmental event registrations. The system demonstrates functional components, JSX, the `useState` hook, event handling, and conditional rendering in a real-world project.

6.2 Future Enhancements

The current system lays a strong foundation but can be significantly enhanced to create a more powerful and scalable platform. Future improvements include:

- **Backend Integration** — Connect with Node.js/Express and MySQL

or MongoDB for persistent booking storage.

- **User Authentication** — Implement JWT-based login to restrict bookings to verified users.
- **QR Code E-Ticket** — Generate a scannable QR code upon successful booking.
- **Email Notifications** — Send automated confirmations using Nodemailer.
- **Cloud Deployment** — Deploy on Vercel or Netlify for public access.

Chapter 7

SOURCE CODE

7.1 Event Configuration Constants

The following section presents the complete source code of the core modules of the TechNova 2025 Ticket Booking Application. The code is organized in the order of dependency — the shared data constants are listed first, followed by the form logic, the display component, and finally the confirmation modal. All files reside inside the `src/` directory of the Vite project. To run the project, place each file in the path indicated above its listing and execute `npm run dev` from the project root.

```
1 // src/eventData.js
2 export const EVENT = {
3   name: "TechNova 2025",
4   tagline: "Annual Technical Symposium",
5   department: "Dept. of Computer Science & Engineering",
6   deptShort: "CSE",
7   date: "Saturday, June 14, 2025",
8   time: "09:00 AM – 06:00 PM IST",
9   venue: "Main Auditorium, Block A",
10  college: "Sri Venkateswara Engineering College",
11  price: 299,
12  totalTickets: 200,
```

```

13 };
14
15 export const DEPARTMENTS = [
16   "CSE", "ECE", "EEE", "Mechanical", "Civil",
17   "IT", "AIDS", "AIML", "CSD", "Faculty", "Other",
18 ];

```

Listing 7.1: eventData.js – Event Configuration Constants

7.2 Form with Full Validation

This is the primary user-facing component. It renders the four-field booking form (Full Name, Email Address, Department dropdown, and Number of Tickets), manages all form state using the `useState` hook, runs the `validate()` function on submission, and calls the `onBook` callback passed down from `App.jsx` when all fields pass validation. It also renders a live total amount preview and a `Reset` button that clears the form without a page reload.

```

1 import { useState } from "react";
2 import { EVENT, DEPARTMENTS } from "../eventData";
3
4 function BookingForm({ available, onBook }) {
5   const [form, setForm] = useState(
6     { name: "", email: "", dept: "", tickets: "" }
7   );
8   const [errors, setErrors] = useState({});
9   const soldOut = available === 0;
10
11   const setField = (key, value) => {
12     setForm((prev) => ({ ...prev, [key]: value }));
13     if (errors[key])
14       setErrors((prev) => ({ ...prev, [key]: "" }));
15   };
16
17   const validate = () => {

```

```

18   const errs = {};
19   if (!form.name.trim())
20     errs.name = "Full name is required.";
21   if (!form.email.trim())
22     errs.email = "Email ID is required.";
23   else if (!/^[\s@]+@[\s@]+\.[^\s@]+$/ .test(form.email))
24     errs.email = "Enter a valid email address.";
25   if (!form.dept)
26     errs.dept = "Please select your department.";
27   if (!form.tickets)
28     errs.tickets = "Number of tickets is required.";
29   else {
30     const n = parseInt(form.tickets, 10);
31     if (isNaN(n) || n < 1)
32       errs.tickets = "Enter a valid positive number.";
33     else if (n > 10)
34       errs.tickets = "Maximum 10 tickets per booking.";
35     else if (n > available)
36       errs.tickets = `Only ${available} ticket(s) left.`;
37   }
38   return errs;
39 };
40
41 const handleSubmit = () => {
42   const errs = validate();
43   if (Object.keys(errs).length > 0) {
44     setErrors(errs); return;
45   }
46   onBook({ ...form, tickets: parseInt(form.tickets, 10) });
47 };
48
49 const handleReset = () => {
50   setForm({ name:"", email:"", dept:"", tickets:"" });
51   setErrors({});
52 };
53
54 return ( /* JSX form with inputs and error spans */ );
55 }
56 export default BookingForm;

```

Listing 7.2: BookingForm.jsx – Form with Full Validation

7.3 Event Information and Availability

This component is responsible for the read-only display side of the application. It renders two cards side by side: an Event Info card that lists the event name, department, date, time, venue, and price badge; and an Availability card that shows the live remaining ticket count, a colour-coded progress bar, and a Sold Out badge when available reaches zero. It takes available and total as props and re-renders automatically whenever available changes in App.jsx after a booking is confirmed.

```
1 import { EVENT } from "../eventData";
2
3 function EventDetails({ available, total }) {
4   const pct = Math.round((available / total) * 100);
5   const soldOut = available === 0;
6
7   return (
8     <div className="event-details-wrap">
9       <div className="card">
10        <p className="section-label">Event Info </p>
11        <div className="info-grid">
12          <InfoItem label="Event" value={EVENT.name} />
13          <InfoItem label="Dept" value={EVENT.deptShort} />
14          <InfoItem label="Date" value={EVENT.date} />
15          <InfoItem label="Time" value={EVENT.time} />
16          <InfoItem label="Venue" value={EVENT.venue} />
17        </div>
18        <div className="price-badge">
19          Rs.{EVENT.price} per ticket
20        </div>
21      </div>
22      <div className="card">
23        <p className="section-label">Availability </p>
24        <span className="ticket-num">{available}</span>
25        <span className="ticket-label">tickets remaining</span>
26        {soldOut && <span className="sold-out">Sold Out</span>}
```

```
27     <div className="progress-bar">
28       <div className="fill" style={{ width: `${pct}%` }}/>
29     </div>
30     <small>{total - available} sold / {pct}% left </small>
31   </div>
32 </div>
33 );
34 }
35 export default EventDetails;
```

Listing 7.3: EventDetails.jsx – Event Information and Availability

Chapter 8

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

8.1 SDG Mapping

Table 8.1: SDG Mapping for the TechNova 2025 Ticket Booking Project

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports digital academic event participation and expands access to technical learning opportunities for students across departments
SDG 9	Industry, Innovation and Infrastructure	Promotes a digital-first campus infrastructure by replacing legacy manual processes with a modern, browser-based web application
SDG 11	Sustainable Cities and Communities	Contributes to smart campus event management by providing efficient, real-time, and inclusive access to institutional events
SDG 12	Responsible Consumption and Production	Eliminates paper-based ticketing and registration forms, reducing paper waste and supporting sustainable administrative practices

As presented in Table 8.1, the application most directly supports **SDG 4 (Quality Education)** by making event participation seamless and inclusive for all students regardless of their department, ensuring equal access to technical fest activities that contribute to experiential and co-curricular learning. **SDG 9 (Industry, Innovation and Infrastructure)** is addressed through the adoption of React JS, Vite, and Razorpay — modern industry-grade tools — within the academic setting, building digital infrastructure competency in a practical project context. **SDG 11** is supported by the smart event management capability: real-time seat tracking, automated overbooking prevention, and instant digital confirmation replace the inefficiencies of manual, error-prone registration. **SDG 12** is supported most tangibly by the complete elimination of paper tickets, registration forms, and manual ledgers, replacing them with a zero-paper digital workflow.

8.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 8.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Diverse technologies	React, Vite, JSX, CSS, Google Fonts
EA2	Level of Interactions	Complex module coupling	Props/callbacks link all four components
EA3	Innovation	Modern React patterns	Hooks-only SPA; no page reload
EA4	Societal Impact	Campus user benefit	Simplifies event participation
EA5	Familiarity	Beyond standard practice	Full-SPA with live validation

As outlined in Table 8.2, **EA1** is satisfied by the deliberate integration of five distinct technologies — React JS, Vite, JSX, custom CSS, and Google Fonts — each serving a specific engineering role. **EA2** reflects the tightly coupled interaction between the four main components (`App.jsx`, `EventDetails`, `BookingForm`, `ConfirmModal`) through props and callback functions, requiring careful design of the data flow to avoid prop-drilling and state inconsistencies. **EA3** highlights the use of the Hooks-only React paradigm, which represents a modern departure from class-based component design and eliminates

page reloads through SPA routing. **EA4** captures the real-world societal benefit of the system for campus communities, while **EA5** acknowledges that a full single-page application with multi-field live validation goes beyond the scope of a basic programming exercise.

8.3 Mapping of Project to Complex Engineering Problems (CEP)

Table 8.3: Mapping of Project to Complex Engineering Problems (CEP)

Level	Attribute	Characteristics	Justification
WP1	Depth of Knowledge	Multiple frontend domains	Components, state, validation, styling
WP2	Conflicting Req.	UX vs. correctness	Smooth form flow with strict validation
WP3	Depth of Analysis	Analytical design	Component boundaries, prop flow design
WP4	Familiarity	Novel campus solution	Custom SPA, not a generic template
WP5	Applicable Standards	React best practices	Functional components, controlled inputs
WP6	Stakeholder Needs	Multiple user types	Students, faculty, and organizers
WP7	Interdependence	Tight coupling	State in App propagates to all children

As shown in Table 8.3, **WP1** is addressed by the requirement to simultaneously master four distinct frontend engineering domains: component design, state management, form validation logic, and CSS-based visual design. **WP2** captures the core engineering tension of the project: balancing a smooth and low-friction user experience against the strict correctness requirements of a ticket booking system where overbooking must never occur. **WP3** reflects the analytical effort required to define correct component boundaries and design a prop flow that avoids unnecessary re-renders. **WP4** demonstrates that the solution is a purpose-built, campus-specific design rather than a generic form template. **WP5** confirms adherence to React community best practices including functional components, controlled form inputs, and single-responsibility component design. **WP6** acknowledges the diverse stakeholder requirements: students need a simple booking experience, faculty need reliable seat tracking, and organisers need data accuracy. **WP7** captures the technical interdependence inherent in the design, where a single state variable (`available`) in `App.jsx` simultaneously drives the display in `EventDetails`, the validation in `BookingForm`, and the confirmation data in `ConfirmModal`.

REFERENCES

- [1] Meta Platforms, Inc. (2024). *React: A JavaScript library for building user interfaces*. **<https://react.dev/>**
- [2] OpenJS Foundation. (2024). *Vite – Next generation frontend tooling*. **<https://vitejs.dev/>**
- [3] OpenJS Foundation. (2024). *Node.js runtime environment documentation*. **<https://nodejs.org/en/docs>**
- [4] BookMyShow. (2023). *Online ticket booking platform for events and entertainment*. **<https://in.bookmyshow.com/>**
- [5] Eventbrite Inc. (2023). *Event management and ticketing platform*. **<https://www.eventbrite.com/>**
- [6] Mozilla Developer Network. (2024). *JavaScript Reference – MDN Web Docs*. **<https://developer.mozilla.org/en-US/docs/Web/JavaScript>**